Taiwo Alabi

Data Scientist. I build AI systems end-to-end and publish exactly where each one fails

alabitaiwo625@gmail.com | +353 87 004 2838 | linkedin.com/in/tami-alabi | github.com/iamtamilore | iamtamilore.github.io | County Dublin D03 K0Y3, Ireland

Profile

I tune intelligence to the cost of being wrong. My coursework and my projects are built around four things: applying software architecture principles to AI systems, evaluating them for completeness and admissibility, critiquing model quality and evaluation design, and critiquing the infrastructure they deploy onto. Before I train anything I establish what the problem looks like solved badly, because **a model that cannot beat guessing the most common answer, or a simple rule, does not deserve to exist**.

Professional Experience

Turing, Remote (Dublin) Apr 2026 to May 2026
AI Quality Analyst, Google Gemini evaluation programme

- Evaluating model output is a labelling problem with no ground truth, so the binding constraint is **consistency between annotators, not correctness against an answer key**. I scored Gemini responses against a written rubric, wrote adversarial prompts to find where the rubric stopped discriminating, and documented the decision rules so a second annotator would agree. Eighty hours of contract work, not a systems role.

Huawei Technologies (via RGS), Globacom Telecommunications department, Lagos Summer 2024 & 2025
Product Manager Intern

- Problem: Telecom mediation moves usage records from network elements to every system that bills, audits or reports on them. 400 million records a day, seven upstream elements fanning out to fourteen downstream consumers, each wanting something different from the same records. The binding constraint is not throughput, it is trust: **a bad deploy means those fourteen systems read nothing, or read twice**, and reading twice bills a customer twice.
- Action: I designed and tested the API layer for a cloud integration between the contact centre platform and a commercial bank's customer care line, tracking every interface the bank had to expose against what we had received and testing each by hand with a command-line tool before it reached a voice prompt. Every branch of the call tree had a recorded prompt, including invalid selection and failed authentication.
- Result: The rule I still use came from the failure path. When authentication fails the call does not retry silently and does not dead-end. **It transfers to a person.** Every system in this document has a version of that rule.
- I also worked with the solutions architect on a data migration that split customer data into separate groups based on what kind of data it was, and with the test team designed over 440 User Acceptance Test cases. During the actual switchover, one storage system ran out of space and blocked new uploads, and a dropped connection interrupted one stream of call records. **Both surfaced in verification, before any customer saw them.**

Selected Projects

Smart Medical Records Engine FastAPI, PostgreSQL/pgvector, LangGraph, Docker

- **Problem:** A doctor has *ten minutes* and a patient history spread across *years of notes*, the one critical line, *a past reaction to a medicine*, gets missed.
- Action: A general vector search across a hospital corpus will eventually surface the wrong patient's record, so **scoping happens at the query layer rather than as a filter applied afterwards**. I tried 30 different ways to make it leak one patient's file into someone else's lookup. None worked. Self-hosting Llama 3.1 costs accuracy against a frontier model, but patient records leaving the network is not a trade a hospital can make.
- **Result:** I ran the *whole stack*, the API, *PostgreSQL with 768-dimensional vector search* and a *self-hosted Llama 3.1*, across *four Docker Compose services*, so *patient data never leaves the hospital's own network*.

  Read the write-up (PDF) | **Architecture diagram** | **Data model** | **Source code**

Customer Care Process Automation Python, scikit-learn, BPMN 2.0

- **Problem:** Hundreds of customer complaints used to get *sorted by hand every morning*, which means we used to have more misrouted tickets because of *human errors*. Customers *waited for minutes* until proper

routing directed their ticket to the right back-end team, followed by a few minutes of communication with that customer to resolve the issue.

- Action: I modelled the process in BPMN 2.0 before writing code, so the automation replaced a process I understood rather than my assumption of one. Field validation has known correct answers, so it went to rules. Reading a complaint and inferring intent does not, so it went to a classifier, with a confidence gate between the classifier and the outcome.
- Result: **The decision was the gate, not the classifier**. Below 0.55 confidence the agent escalates to a human rather than forcing a label, and I report the lower automation rate that produces. Naive Bayes was chosen over heavier models because 210 tickets will overfit anything larger, and because a misroute has to be inspectable.

  Read the write-up (PDF) | BPMN process models | Confusion matrix | Source code

TermsGuard AI Sentence transformers, FAISS, Claude API, Flask

- Problem: A 2008 study estimated that reading every privacy policy an average internet user meets would take about 244 hours a year. A 2017 Deloitte survey of 2,000 consumers found 91% accept terms without reading them.
- Action: The failure mode was hallucination, so every flag is grounded in GDPR text retrieved from a 4,527-chunk index. Cosine similarity alone marks "we do not sell your data" as a violation, because **the model can barely tell "sell" from "do not sell" apart**, so rule guards sit on top of the score rather than instead of it.
- **Results:** I cut *end-to-end latency from 28 seconds to under 10*, and added a *Claude verification stage* that removes false positives, and built a cosine-only fallback when no API key is set.

  Live demo | Read the write-up (PDF) | Source code

Automated QA Gate for AI Generated Imagery TensorFlow/Keras, ResNet50, scikit-learn

- **Problem:** *AI-generated fashion campaign images* silently drift over multiple prompt generations; the character's *identity starts to shift* or specific details I want to retain become inconsistent.
- Action: Two baselines first. Pixel difference reached F1 0.59, kNN 0.26, the Siamese network 0.90 with precision 1.00 on a 90-pair test set. **That gap is the result; 0.90 on its own is not.** I moved the decision cutoff away from the model's default halfway point, 0.5, to 0.4821, tuned only on validation data, because a rejected good image costs one regeneration and an accepted bad one reaches the client. This only catches identity drift, not the colour and lighting drift a batch can also suffer, which needs a separate, weaker autoencoder check (62% catch rate).

  Read the write-up (PDF) | ROC curve and confusion matrix | Architecture diagram | Source code

Education

National College of Ireland, Dublin Jan 2026 to Jan 2027
MSc Artificial Intelligence (part time)

MSc Artificial Intelligence (part time). Assessed on reviewing and critiquing AI systems, not only building them: architecture review against quality attributes, evaluating search for completeness and admissibility, critiquing model quality and evaluation design, and critiquing deployment infrastructure.

Afe Babalola University, Lagos Sep 2021 to Feb 2025
BSc Computer Science, Second Class Honours, Upper Division (2.1)

Technical Skills

Languages: Python, SQL

Data & Infrastructure: Hadoop HDFS, HBase, GaussDB, PostgreSQL, pgvector, Docker, Docker Compose, Kubernetes, FastAPI, Flask, Google Cloud

Machine Learning: scikit-learn, pandas, TensorFlow/Keras, transfer learning, reinforcement learning, Q-Learning, neural networks, Transformers

AI & Agents: RAG pipelines, vector search (pgvector, FAISS), LangGraph, LLM evaluation, genetic algorithms

Systems and Evaluation: training-serving skew and feature contracts, model registries and versioned artefacts, blue-green and canary rollout, decoupling inference from the OLTP path, p95 and p99 latency targets with defined fallback, baseline design, sliced evaluation, confidence calibration, GDPR data cataloguing, data lineage, UAT design.

Tools: Git, GitHub, Claude Code, Linux